

Cost Manager – RESTful Web Services (Final Project)

This project is implemented as **four independent microservices**, each running in its own process and on its own port, according to the course requirements.

In addition, the project includes **unit tests** that are executed from the root folder and communicate with the running services via real HTTP requests.

Project Structure

```
node.js-final-project/
├── services/
│   ├── logs-service/      (port 3001)
│   ├── users-service/     (port 3002)
│   ├── costs-service/     (port 3003)
│   └── admin-service/     (port 3004)
├── tests/                 (unit tests for all services)
├── package.json           (root scripts for running all services & tests)
└── README.md
```

Microservices Overview

Service	Responsibility	Port
logs-service	Retrieve system logs	3001
users-service	Users management (add user, list users, user details)	3002
costs-service	Add cost items, generate monthly report	3003
admin-service	Developers team information (/api/about)	3004

Each service is a **standalone Node.js + Express + MongoDB application**.

Running Each Microservice Individually

You may run each service independently from its own folder.

Example (users-service):

```
cd services/users-service  
npm install  
npm start
```

Repeat for any other service:

```
services/logs-service  
services/costs-service  
services/admin-service
```

Each service will start its own server and connect to MongoDB.

Running All Services Together (from Root Folder)

The root project includes scripts that start all microservices simultaneously using **concurrently**.

One-time setup

From the root folder:

```
npm install
```

Then install dependencies for each service:

```
npm --prefix services/logs-service install  
npm --prefix services/users-service install  
npm --prefix services/costs-service install  
npm --prefix services/admin-service install
```

Start all services

```
npm start
```

or

```
npm run start:all
```

Expected output:

```
logs-service running on http://localhost:3001  
users-service running on http://localhost:3002
```

```
costs-service running on http://localhost:3003  
admin-service running on http://localhost:3004
```

To stop all services:

```
Ctrl + C
```

API Quick Check

You can verify that services are running using a browser or Postman:

- <http://localhost:3001/api/logs>
- <http://localhost:3002/api/users>
- <http://localhost:3003/api/report?id=123123&year=2025&month=1>
- <http://localhost:3004/api/about>

Unit Tests

Unit tests are located in:

```
/tests
```

Each service has its own test file:

- `logs.test.js`
- `users.test.js`
- `costs.test.js`
- `admin.test.js`

Important notes

- Tests send **real HTTP requests** to the running services.
- Tests use the **same MongoDB database** as the project.
- Tests may create temporary users and cost items during execution.
- Tests are symbolic and verify:
 - Endpoint availability
 - HTTP status codes
 - Basic response structure

Running tests

Make sure all services are running first, then from root:

```
npm test
```

Environment Variables

Each microservice uses its own `.env` file for:

- MongoDB connection string
 - Port configuration
-

Database Requirements

When submitting the project, the database must be empty except for one user:

```
id: 123123
first_name: mosh
last_name: israeli
```

Technologies Used

- Node.js
 - Express.js
 - MongoDB Atlas
 - Mongoose
 - Pino (logging)
 - Jest + Axios (unit tests)
 - concurrently (multi-process execution)
-

Notes

- Each microservice is fully independent and can be deployed separately.
 - Root folder scripts are provided only for development convenience.
 - Unit tests are for code review purposes as defined by course requirements.
-

Good luck 🚀